

Jzzhu and Cities

Caminhos mínimos com Dijkstra e análise de arestas redundantes

Codeforces 449B • Grupo F • Python 3

O problema

n cidades, capital = cidade 1. Há **estradas** bidirecionais (peso x) e **rotas de trem** ligando a capital a uma cidade (peso y).

Queremos fechar o **máximo de rotas de trem** sem que a distância mínima de nenhuma cidade até a capital mude.

Modelagem como grafo

- **Vértices:** as n cidades; origem $s = 1$.
- **Arestas-estrada:** (u, v) bidirecional, peso x.
- **Arestas-trem:** (1, c) bidirecional, peso y.
- Grafo não-direcionado $\Rightarrow$ distância cidade $\rightarrow$ capital = capital $\rightarrow$ cidade.
- Pesos ≥ 1 (não negativos) $\Rightarrow$ Dijkstra é aplicável.

Restrição que define a solução

Um trem para a cidade c só é necessário quando é a **única** forma de manter a distância mínima de c. Caso contrário, pode ser fechado.

Limites

- $n \leq 10^5$; $m \leq 3 \cdot 10^5$; $k \leq 10^5$
- $x, y \leq 10^9$ (somas cabem em inteiros grandes).
- Conectividade garantida pelo enunciado.

Um único Dijkstra a partir da capital

Rodamos Dijkstra sobre estradas + a **menor** rota-trem de cada cidade. A fila de prioridade mínima escolhe sempre o vértice de menor `distTo[]`; cada aresta é relaxada uma vez.

Decisão de cada rota-trem (peso y para a cidade c):

- **Por cidade só a menor rota importa** — as demais já são fechadas.
- $y > \text{dist}[c]$ → o trem nem atinge o mínimo → fecha.
- $y = \text{dist}[c]$ e alguma **estrada** atinge `dist[c]` → redundante → fecha.
- $y = \text{dist}[c]$ e nenhuma estrada atinge `dist[c]` → única forma → mantém.

A contagem de estradas que realizam `dist[c]` (`cnt_road[c]`) é feita numa varredura $O(V+E)$ após o Dijkstra.

```
relax: if dist[v] + peso(v,w) < dist[w]: dist[w] = dist[v] + peso(v,w)
```

Complexidade

$V = n$, $E = m$, k rotas de trem.

- Construção + pré-filtragem dos trens: $O(V+E+k)$.
- Dijkstra (heap binário): $O((V+E) \log V)$.
- Varredura final e decisão: $O(V+E+k)$.

Tempo total: $O((V+E) \log V + k)$.

Espaço: $O(V+E)$ (grafo em CSR + vetores).

Casos especiais

- Estradas/trens paralelos: mantém-se a melhor de cada.
- Empate trem = estrada: fecha o trem.
- Vários trens mínimos iguais sem estrada: mantém só um.
- Pesos grandes ($\leq 10^9$): inteiros do Python, sem overflow.
- Toda cidade alcança a capital (sem inalcançáveis).

Validação: 5069 casos contra força-bruta que testa todos os subconjuntos de trens — 0 divergências.

Resumo

- Dijkstra padrão sobre o grafo combinado resolve as distâncias.
- A pergunta vira uma análise local: cada trem é necessário só se for a única forma de manter $\text{dist}[c]$.
- Solução $O((V+E) \log V + k)$, dentro do limite de 2 s mesmo em Python.

Por que Dijkstra (e não Bellman-Ford)?

Todos os pesos são não-negativos, então não há ciclos negativos; Dijkstra é correto e mais rápido — $O((V+E) \log V)$ vs $O(V \cdot E)$.

Obrigado!